Intelligent Real Estate Advisory System

ML Prediction, RAG, and LangGraph Agent Workflow
Ames Housing Dataset

Kavya Katal Aabir Sarkar Kushagra Maheswari Pratyush Chouksey

April 21, 2026

Abstract

This project presents an intelligent real estate advisory system that integrates Machine Learning (ML), Retrieval-Augmented Generation (RAG), and an agentic workflow orchestrated by LangGraph. The system predicts residential property prices from the Ames Housing dataset using a scikit-learn Linear Regression pipeline with a log-transformed target, achieving an R^2 of 0.93 and MAE of $14,899 on the validation set. Beyond numerical prediction, a three-node LangGraph agent enriches the output by retrieving market context from a ChromaDB vector store via sentence-transformer embeddings, performing comparable-property analysis against the historical dataset, and generating a structured advisory report through an LLM served by OpenRouter. A deterministic rule-based fallback guarantees advisory availability when the LLM is unreachable. The entire pipeline is deployed as an interactive Streamlit web application, providing users with price estimates, market insights, and investment recommendations in a single interface.

Contents

Part I
Machine Learning Pipeline

1 Problem Statement

Buying or selling a house involves high-stakes financial decisions. However, manually estimating a fair market value is difficult because property prices depend on a large number of factors such as neighbourhood, living area, construction quality, age, and amenities. Traditional appraisal methods are labour-intensive, subjective, and hard to scale.

Goal: Given structured information about a residential property, we aim to automatically predict its sale price (in USD) using supervised machine learning. The model should:

- Produce **reasonably accurate and stable** price estimates on unseen properties.
- **Generalise** beyond the training data without overfitting.
- Be **fast and lightweight** enough to support an interactive web interface.
- Be **interpretable and maintainable**, with a clear preprocessing pipeline and reproducible training process.

Scope: The project focuses on predicting prices for properties in the Ames, Iowa housing market using historical sales data. While the deployed Streamlit app exposes only a subset of features to the user (e.g., overall quality, living area, basement area, year built), the underlying training pipeline handles the full feature set of the dataset.

2 Data Description

2.1 Dataset Origin

We use the **Ames Housing** dataset, popularised via the Kaggle competition "House Prices: Advanced Regression Techniques". The dataset contains detailed information about residential property sales in Ames, Iowa.

- **Source:** Kaggle / Ames Housing public dataset.
- **File used:** A large tabular CSV (referred to as `BigHousing.csv` in this repository).
- **Target variable:** `SalePrice` (USD).

2.2 Feature Overview

The raw dataset contains around 80 input features describing each property. These features fall into the following broad categories:

- **Physical characteristics**

  - `GrLivArea`: Above-ground living area (square feet).
  - `TotalBsmtSF`: Basement area (square feet).
  - `LotArea`: Lot size (square feet).
  - `BsmtFullBath`, `FullBath`: Number of bathrooms.
  - `GarageCars`: Garage capacity (number of cars).
  - `Fireplaces`: Number of fireplaces.

- **Quality and condition**
  - `OverallQual`: Overall material and finish quality (1–10).
  - `OverallCond`: Overall condition rating.
  - Exterior/interior material quality ratings (e.g. `ExterQual`, `KitchenQual`).

- **Age and year-related variables**
  - `YearBuilt`: Original construction date.
  - `YearRemodAdd`: Remodel date.
  - `YrSold`: Year sold.

- **Location and configuration**
  - `Neighborhood`: Physical location within Ames.
  - `MSZoning`: General zoning classification.
  - `HouseStyle`, `BldgType`: Building and house style.

In our training pipeline, we automatically infer column types:

- **36 numeric** columns (floats/integers).
- **43 categorical** columns (strings or categories).

The `Id` column is an identifier and is dropped before training.

2.3 Target Variable

The target is the final sale price of each property:

- **Name:** `SalePrice`
- **Unit:** USD
- **Type:** Continuous numeric

As explored in Section 3.3, `SalePrice` is right-skewed, motivating a log-transformation during model training.

3 EDA Process

Our exploratory data analysis (EDA) was performed primarily in a Jupyter notebook and guided the preprocessing and modelling choices.

3.1 Dataset Audit and Structure

We first examined the basic structure of the dataset:

- Checked the shape: approximately 1460 rows and 80+ features.
- Inspected column data types to separate numeric and categorical variables.
- Verified that the `SalePrice` column (target) exists and has no missing values.
- Confirmed the presence of both continuous measurements (e.g., square footage) and many categorical descriptors (e.g., neighbourhood, exterior material).

3.2 Missing Values and Data Quality

We computed the count of missing values per column and sorted them in descending order. Several features exhibited substantial missingness, especially optional attributes such as `Alley`, `PoolQC`, and some basement/garage features.

3.3 Target Distribution and Transformation

The raw distribution of `SalePrice` is strongly right-skewed, with a long tail of expensive properties. This violates the normality assumption underlying many regression models and can cause models to underperform on typical houses or be overly influenced by outliers.

Using the notebook, we visualised the histogram of `SalePrice` and then the histogram of $\log(1 + \texttt{SalePrice})$. The log-transformed target is much closer to a symmetric, approximately normal distribution, which stabilises training and improves model fit.

3.4 Correlation Analysis

We computed the Pearson correlation between numeric features and the target (in USD) and plotted the 12 most correlated numeric features.

Key observations:

- **Overall quality (`OverallQual`)** has one of the strongest positive correlations with `SalePrice`.
- **Living area (`GrLivArea`)** and **basement area** (`TotalBsmtSF`) are also highly correlated.
- **Garage capacity** (`GarageCars`) and **year built** (`YearBuilt`) show meaningful positive correlations.

These insights motivated both the choice of input features highlighted in the Streamlit UI and the expectation that a mostly linear model might perform well after appropriate transformations.

3.5 Train–Validation Split

To obtain unbiased estimates of model performance, we split the dataset into training and validation sets:

- **Train:** 80% of the data.
- **Validation:** 20% of the data.
- **Random seed:** 42 (for reproducibility).

This split results in 1168 training examples and 292 validation examples. The same split is used for all model comparisons to ensure fairness.

4 Methodology

4.1 Overall Pipeline Design

Instead of manually cleaning and transforming features in an ad hoc manner, we use a **scikit-learn pipeline** based on `ColumnTransformer` and `Pipeline` objects. This design has several advantages:

- Training and inference (in the Streamlit app) run the **exact same** preprocessing steps.
- Complex transformations are encapsulated and reproducible.
- The final model artifact can be serialised with joblib and loaded directly in production.

The high-level flow is:

1. Load and clean raw data.
2. Separate features $\mathbf{X}$ and target $y = \texttt{SalePrice}$.
3. Drop the `Id` column.
4. Split into training and validation sets.
5. Apply $\log(1 + y)$ transformation to the target for training.
6. Fit preprocessing + model pipelines on training data.
7. Evaluate on the validation set, converting predictions back to USD using $\exp(y_{\log}) - 1$.
8. Save the final pipeline, target transform flag, and training column names as a joblib artifact.

4.2 Preprocessing for Numeric Features

Numeric features are processed through the following steps:

- **Missing value imputation:** Replace missing values with the median of each column using `SimpleImputer(strategy="median")`.
- **Scaling (for Linear Regression):** Standardise features to zero mean and unit variance using `StandardScaler`.

For the Random Forest model, scaling is not required, so we only perform median imputation.

4.3 Preprocessing for Categorical Features

Categorical features undergo:

- **Most-frequent imputation:** Fill missing values with the most frequent category per column.
- **One-hot encoding:** Convert categorical values into binary indicator columns using `OneHotEncoder(handle`

Using `handle_unknown="ignore"` ensures that the model can safely process categories that did not appear during training (important in deployment).

4.4 Target Transformation

As established in Section 3.3, the target `SalePrice` is right-skewed. We therefore train models on:

$$y_{\log} = \log(1 + \texttt{SalePrice})$$

At prediction time, we invert this transformation to obtain prices in USD:

$$\widehat{\texttt{SalePrice}} = \exp(y_{\log}^{\text{pred}}) - 1$$

This transformation reduces the influence of extreme outliers, brings the target distribution closer to normal, and often improves both MAE and RMSE for regression tasks with heavy-tailed targets. The transformation flag (`"log1p"`) is stored with the saved artifact so the Streamlit app can consistently apply the inverse transform via `numpy.expm1`.

4.5 Baseline Model: Linear Regression

We first build a baseline using ordinary least squares Linear Regression:

- **Model:** scikit-learn's `LinearRegression`.
- **Pipeline:** Numeric: median imputation + standard scaling. Categorical: most-frequent imputation + one-hot encoding. Regressor: Linear Regression trained on $y_{\log}$.

4.6 Comparison Model: Random Forest Regressor

To capture potential non-linear relationships and interactions, we also train a Random Forest Regressor:

- **Model:** `RandomForestRegressor` with `n_estimators = 600`, `random_state = 42`, `n_jobs = -1`, and default depth.
- **Pipeline:** Numeric: median imputation (no scaling). Categorical: most-frequent imputation + one-hot encoding. Regressor: Random Forest trained on $y_{\log}$.

4.7 Model Serialization and Deployment Artifacts

After training, we save the artifacts using joblib:

- **Pipeline:** The final trained Linear Regression pipeline (preprocessing + model).
- **Target transform flag:** `"log1p"` to indicate the target was transformed.
- **Training columns:** The list of original feature columns expected by the pipeline.

These are stored in a single joblib file (`model/house_price_lr_pipeline.joblib`) which is loaded by the Streamlit app.

5 ML Model Evaluation

5.1 Metrics

We evaluate regression performance on the validation set using:

- **MAE (Mean Absolute Error):**

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- **RMSE (Root Mean Squared Error):**

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- **R^2 (Coefficient of Determination):**

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

All metrics are reported in the original price space (USD), after inverting the log-transform.

5.2 Quantitative Results

Table 1 summarises the performance of both models on the validation set.

Table 1: Validation performance of Linear Regression and Random Forest models.

Model	**MAE (USD)**	**RMSE (USD)**	$\mathbf{R^2}$
Linear Regression (log target)	14,899	22,740	0.9326
Random Forest (log target)	17,495	30,153	0.8815

5.3 Interpretation

Despite being simpler, the Linear Regression model achieves a lower MAE and RMSE compared to the Random Forest, and explains a higher proportion of variance in prices ($R^2 \approx 0.93$ vs. 0.88). These results suggest that, after appropriate preprocessing and log-transforming the target, the relationship between the selected features and `SalePrice` is largely linear. The more complex Random Forest model underperforms, likely because it overfits noise and does not benefit enough from modelling non-linearities at this dataset scale.

5.4 Model Selection

Based on the validation metrics and practical concerns (speed, interpretability, and ease of deployment), we select the **Linear Regression (log target)** pipeline as the final model for deployment.

Part II
Retrieval-Augmented Generation (RAG)

6 Motivation for RAG

While the ML model provides a numerical price prediction, real estate decisions require additional contextual reasoning: market trends, investment heuristics, risk factors, and domain knowledge. Rather than embedding all this knowledge directly into the LLM prompt (which would be static, token-expensive, and difficult to maintain), we adopt a **Retrieval-Augmented Generation (RAG)** architecture.

RAG separates knowledge storage from reasoning:

1. A **knowledge base** of domain-specific documents is embedded into a vector store at ingestion time.
2. At query time, the user's property context is encoded into the same embedding space, and the most semantically relevant documents are retrieved.
3. The retrieved context is injected into the LLM prompt, grounding the model's advisory in factual, retrievable evidence.

This architecture provides several advantages over a pure LLM approach:

- **Grounded responses:** The LLM reasons over retrieved evidence rather than relying solely on parametric memory, reducing hallucination.
- **Updatable knowledge:** New insights can be added to the vector store without retraining or modifying the LLM.
- **Transparency:** Retrieved documents serve as citable evidence, making the advisory auditable.
- **Cost efficiency:** Only the most relevant context is sent to the LLM, keeping token usage low.

7 RAG Architecture

7.1 System Diagram

Figure 1 illustrates the RAG pipeline from knowledge ingestion through retrieval to prompt augmentation.

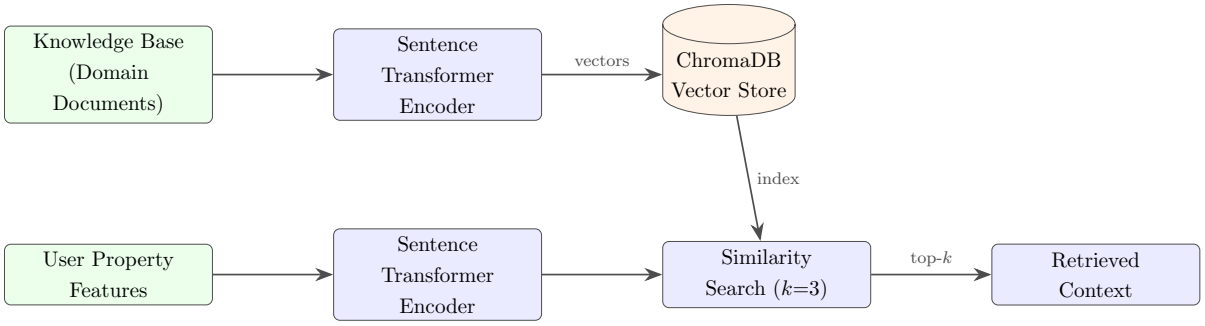

Figure 1: RAG pipeline: offline ingestion (top) and online retrieval (bottom).

7.2 Embedding Model

We use the **sentence-transformers/all-MiniLM-L6-v2** model from the Sentence Transformers library, accessed through LangChain's `HuggingFaceEmbeddings` wrapper.

Key properties of this embedding model:

- **Dimensionality:** 384-dimensional dense vectors.
- **Architecture:** Distilled BERT variant fine-tuned for semantic similarity tasks.
- **Speed:** Lightweight enough to run on CPU without GPU requirements.
- **Quality:** Competitive performance on semantic textual similarity benchmarks, making it well-suited for matching property queries against domain knowledge.

7.3 Vector Store: ChromaDB

ChromaDB serves as the persistent vector database:

```python
from langchain_community.vectorstores import Chroma
from langchain_huggingface import HuggingFaceEmbeddings

def get_vectorstore():
    embedding = HuggingFaceEmbeddings(
        model_name="sentence-transformers/all-MiniLM-L6-v2"
    )
    vectordb = Chroma(
        persist_directory="db",
        embedding_function=embedding,
    )
    return vectordb
```

Listing 1: Vector store initialisation (`src/rag/vectorstore.py`).

ChromaDB stores embeddings on disk in the `db/` directory, providing persistence across application restarts without re-embedding the knowledge base.

7.4 Knowledge Base and Ingestion

The knowledge base consists of curated real estate domain insights. These documents encode investment heuristics, property valuation rules, and risk considerations:

```
docs = [
    "High quality houses appreciate faster.",
    "Large living area increases property value.",
    "Newer homes have better resale potential.",
    "Investment depends on market demand.",
    "Always verify legal documents before buying.",
]

vectordb = Chroma.from_texts(
    texts=docs,
    embedding=embedding,
    persist_directory="db",
)
```

Listing 2: Knowledge base ingestion (`src/rag/ingest.py`).

The ingestion script embeds each document string into a 384-dimensional vector and stores it in ChromaDB along with the original text. An extended knowledge base is also defined in `src/rag/knowledge_base.py` with additional domain rules covering investment risk levels, neighbourhood preferences, and due diligence guidance.

7.5 Retrieval

At query time, the retriever constructs a query string from the user's property features and performs a cosine similarity search against the vector store:

```
from src.rag.vectorstore import get_vectorstore

def retrieve_context(query):
    db = get_vectorstore()
    docs = db.similarity_search(query, k=3)
    return [d.page_content for d in docs]
```

Listing 3: Context retrieval (`src/rag/retriever.py`).

The retriever returns the top $k = 3$ most semantically similar documents. These are plain text strings that are later injected into the LLM prompt as "Market Context." The choice of $k = 3$ balances context richness against prompt length and token cost.

Part III

Agent Workflow (LangGraph)

8 Agent Architecture Overview

8.1 Why an Agentic Approach?

A traditional ML prediction provides a single number. Real estate advisory, however, requires multiple reasoning steps: contextualising the prediction against comparable properties, incorporating market knowledge, assessing valuation, and synthesising a structured report.

We model this multi-step reasoning as an **agent workflow** using **LangGraph**, a library for building stateful, graph-based AI applications. Each reasoning step is implemented as a **node** in a directed acyclic graph (DAG), with a shared **state** object flowing through the graph.

8.2 Design Principles

- **Separation of concerns:** Each node has a single responsibility (retrieval, analysis, or generation).
- **Stateful execution:** The `AgentState` TypedDict accumulates outputs across nodes, ensuring downstream nodes have access to all prior results.
- **Deterministic topology:** The graph follows a fixed linear path, providing predictable execution order and debuggability.
- **Graceful degradation:** If the LLM API fails, a rule-based fallback generates the advisory report deterministically.

9 Agent State Schema

The agent communicates through a typed state dictionary that is passed between nodes:

```python
from typing import TypedDict, List, Dict

class AgentState(TypedDict):
    user_input: Dict[str, float]
    predicted_price: float
    comparable_summary: str
    market_context: List[str]
    regulation_context: List[str]
    investment_goal: str
    risk_level: str
    reasoning: str
    recommendation: str
    final_report: str
```

Listing 4: Agent state definition (`src/agent/state.py`).

Table 2 describes each field and which node produces it.

Table 2: Agent state fields, their sources, and descriptions.

Field	Producer	Description
`user_input`	App (pre-agent)	Property feature values from the UI
`predicted_price`	App (pre-agent)	ML model prediction in USD
`comparable_summary`	App (pre-agent)	Summary of comparable properties
`market_context`	Node 1	RAG-retrieved domain insights
`regulation_context`	Reserved	Placeholder for regulatory data
`investment_goal`	App (pre-agent)	Default investment intent (currently hardcoded)
`risk_level`	App (pre-agent)	Default risk setting (currently hardcoded)
`reasoning`	Node 2	Valuation analysis explanation
`recommendation`	Node 2	Valuation flag (over/under/fair)
`final_report`	Node 3	Complete advisory report text

10 Graph Topology and Node Definitions

10.1 Graph Construction

The agent graph is constructed using LangGraph's `StateGraph` API:

```python
from langgraph.graph import StateGraph, START, END
from src.agent.state import AgentState
from src.agent.langgraph_nodes import (
    retrieve_market_context_node,
    analyze_property_node,
    generate_report_node,
)

def build_agent_graph():
    graph = StateGraph(AgentState)

    graph.add_node("retrieve_market_context",
                   retrieve_market_context_node)
    graph.add_node("analyze_property",
                   analyze_property_node)
    graph.add_node("generate_report",
                   generate_report_node)

    graph.add_edge(START, "retrieve_market_context")
    graph.add_edge("retrieve_market_context",
                   "analyze_property")
    graph.add_edge("analyze_property",
                   "generate_report")
    graph.add_edge("generate_report", END)

```

```
    return graph.compile()
```

Listing 5: Graph definition (`src/agent/graph.py`).

Figure 2 visualises the execution flow.

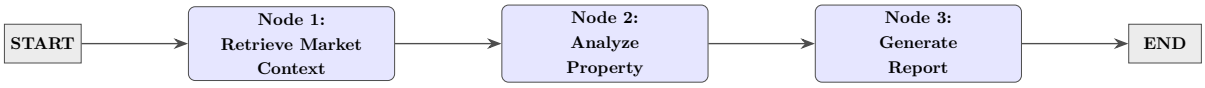

Figure 2: LangGraph agent: three-node linear DAG.

10.2 Node 1: Retrieve Market Context

This node invokes the RAG retriever to fetch domain knowledge relevant to the user's property:

```
def retrieve_market_context_node(state):
    query = f"Property features: {state['user_input']}"
    context = retrieve_context(query)
    return {"market_context": context}
```

Listing 6: Retrieve market context node.

The node constructs a query string from the user's input features (e.g., `OverallQual`, `GrLivArea`, etc.), passes it to the RAG retriever described in the previous part, and stores the top-3 retrieved documents in the `market_context` field.

10.3 Node 2: Analyze Property

This node performs a rule-based valuation assessment by comparing the predicted price against comparable properties:

```
def _get_valuation_flag(price, comps):
    avg_price = _extract_avg_price(comps)
    if avg_price is None or avg_price <= 0:
        return "unknown"
    if price > avg_price * 1.2:
        return "overvalued"
    if price < avg_price * 0.8:
        return "undervalued"
    return "fair"

def analyze_property_node(state):
    valuation_flag = _get_valuation_flag(
        state["predicted_price"],
        state["comparable_summary"]
    )
    reasoning = (
        f"Predicted price analyzed against comparables. "
        f"Valuation signal: {valuation_flag}."
```

```
    )
    return {
        "reasoning": reasoning,
        "recommendation": valuation_flag,
    }
```

Listing 7: Analyze property node with valuation logic.

The valuation logic uses a ±20% threshold around the comparable average price:

- **Overvalued**: Predicted price exceeds 120% of the comparable average.
- **Undervalued**: Predicted price falls below 80% of the comparable average.
- **Fair**: Predicted price is within the 80–120% band.
- **Unknown**: Comparable data is insufficient for assessment.

10.4 Node 3: Generate Report

The final node synthesises all accumulated state into a structured advisory report. It first attempts to generate the report via the LLM; if that fails, it falls back to a deterministic rule-based report.

10.4.1 LLM-Based Generation

The node constructs a detailed prompt that includes the predicted price, comparable analysis, retrieved market context, and valuation signal. The prompt enforces a strict output format with sections for Property Valuation, Market Insights, Comparable Analysis, Recommendation, Risks, and Disclaimer. The LLM is instructed to use only the provided data and to avoid hallucination.

10.4.2 Rule-Based Fallback

If the LLM call fails (due to API unavailability, timeout, or empty response), the node generates a deterministic report using the valuation flag:

- **Overvalued** → Recommendation: *Caution*
- **Undervalued** → Recommendation: *Buy*
- **Fair / Unknown** → Recommendation: *Wait*

This fallback ensures the system always produces an advisory, maintaining user trust even when the external LLM API is unavailable.

11 LLM Integration

11.1 OpenRouter API

The system uses **OpenRouter** as the LLM provider, which acts as a unified gateway to multiple language models. The integration is implemented via direct HTTP requests:

```python
@lru_cache(maxsize=50)
def get_llm_response(prompt: str) -> str:
    api_key = os.getenv("OPENROUTER_API_KEY")
    url = "https://openrouter.ai/api/v1/chat/completions"
    data = {
        "model": "openrouter/free",
        "messages": [
            {"role": "system",
             "content": "You are a grounded real estate "
                        "investment advisor..."},
            {"role": "user", "content": prompt},
        ],
        "temperature": 0.2,
    }
    response = requests.post(url, headers=headers,
                             json=data, timeout=60)
    return content.strip()
```

Listing 8: LLM integration (`src/agent/llm.py`).

Key design decisions:

- **Low temperature** (0.2): Prioritises factual, consistent outputs over creative responses.
- **System prompt grounding**: The system message instructs the LLM to use only provided data, preventing hallucination of legal facts or market guarantees.
- **Response caching**: `@lru_cache` avoids redundant API calls for identical prompts within a session.
- **Timeout handling**: A 60-second timeout prevents indefinite blocking.

12 Comparable Property Analysis

Before the agent runs, the application performs a comparable property analysis against the historical dataset. This provides the `comparable_summary` field that Node 2 and Node 3 both rely on:

```python
def get_comparables(user_input, df):
    filtered = df.copy()
    filtered = filtered[
        (abs(filtered["GrLivArea"] - user_input["GrLivArea"]) < 500)
            &
```

```
        (abs(filtered["OverallQual"] - user_input["OverallQual"]) <=
            1)
    ]
    if filtered.empty:
        return "No strong comparable properties found."
    avg_price = filtered["SalePrice"].mean()
    min_price = filtered["SalePrice"].min()
    max_price = filtered["SalePrice"].max()
    return f"""
    Based on similar properties:
    - Avg Price: ${avg_price:,.0f}
    - Range: ${min_price:,.0f} - ${max_price:,.0f}
    """
```

Listing 9: Comparable analysis (`src/core/comparable.py`); excerpt shown with original return format.

The filter selects properties with living area within ±500 sq. ft. and overall quality within ±1 grade of the target property, then reports the average, minimum, and maximum sale prices of the matched properties.

Part IV

Integrated System and Results

13 End-to-End Pipeline

Figure 3 shows the complete system pipeline, from user input through ML prediction, comparable analysis, RAG retrieval, agent reasoning, and final advisory output.

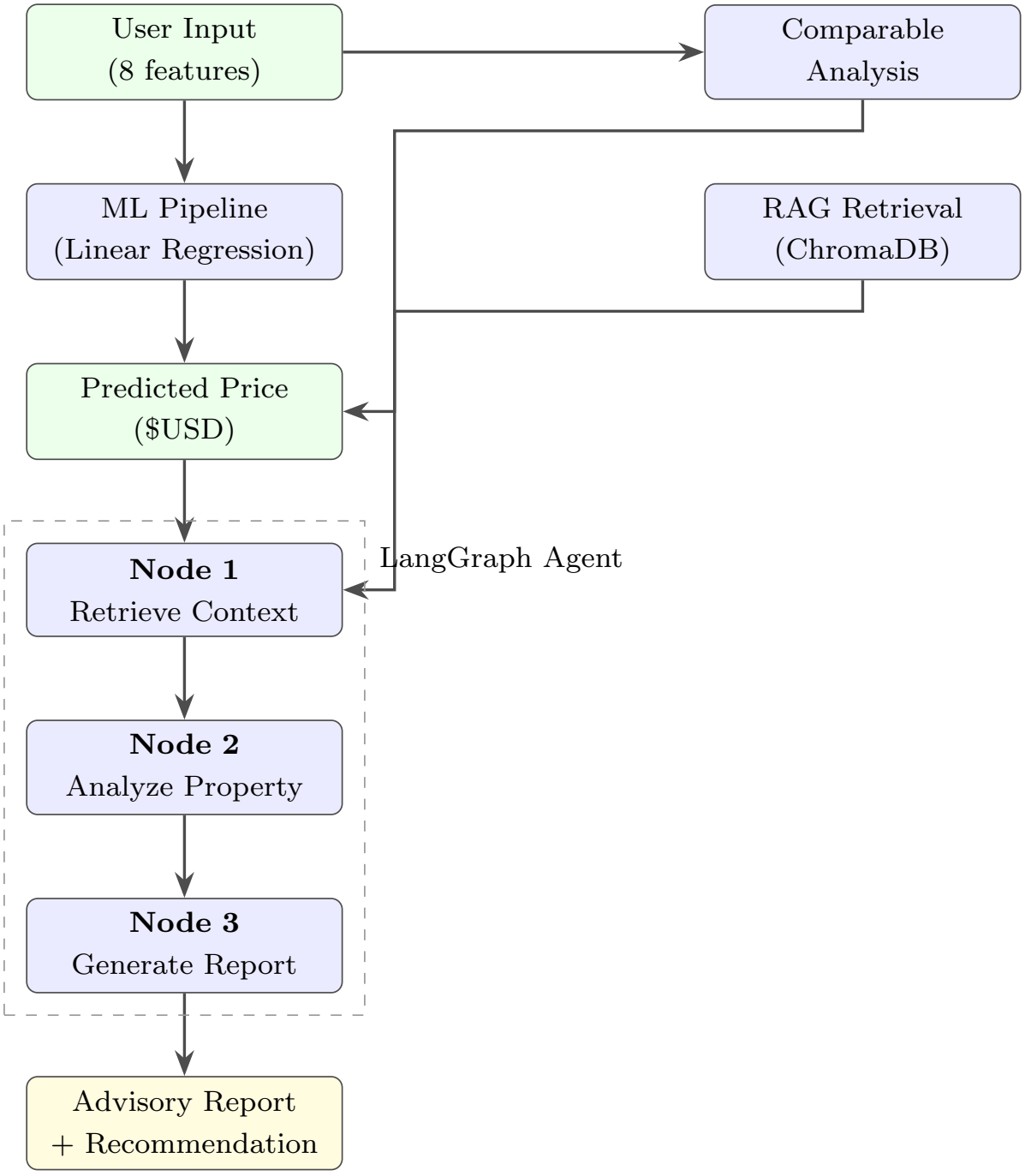

Figure 3: End-to-end system pipeline from user input to advisory report.

The pipeline executes in four sequential stages:

1. **ML Prediction (Step 1):** The user's feature inputs are transformed into a single-row DataFrame matching the training schema, then passed through the loaded scikit-learn pipeline to produce a price estimate in USD.
2. **Comparable Analysis (Step 2):** The user's features are matched against the historical dataset to find similar properties and compute summary statistics.
3. **Agent Execution (Step 3):** The LangGraph agent runs its three nodes sequentially—retrieving market context, analysing valuation, and generating the advisory report.
4. **UI Rendering (Step 4):** The Streamlit frontend displays the predicted price, advisory report, evidence panel (retrieved context and comparables), and prediction history.

14 Streamlit Application

14.1 User Interface

The Streamlit app provides a two-column layout:

- **Left column:** A prediction card showing the latest estimated price, input controls (sliders and number inputs) for eight key features, and Predict/Clear action buttons.
- **Right column:** A prediction history chart, the AI-generated advisory report (with sections for Valuation, Market Insights, Comparable Analysis, Recommendation, Risks, and Disclaimer), and a history table.

The application also provides **preset configurations** (Starter, Family, Premium) that prefill the input controls with representative property profiles, enabling quick exploration.

14.2 Evidence and Transparency

The UI includes an evidence panel that shows:

- **Retrieved market context:** The RAG-sourced domain insights used in the advisory.
- **Comparable properties:** The statistical summary of matched historical sales.

This transparency allows users to understand and verify the basis of the AI's recommendation.

15 Integrated Results and Discussion

15.1 ML Model Performance

As reported in Section 5, the Linear Regression model with log-transformed target achieves strong predictive performance:

- **R^2 = 0.9326**: The model explains 93.3% of the variance in sale prices.
- **MAE = $14,899**: On average, predictions deviate by approximately $15K from actual prices, which is reasonable for the Ames housing market where median prices are approximately $160K.
- **RMSE = $22,740**: Larger errors (e.g., for luxury properties) contribute more to RMSE, but the overall magnitude remains acceptable.

15.2 RAG Retrieval Quality

The RAG system retrieves contextually relevant domain knowledge that grounds the advisory. For a typical query about a high-quality, large-area property, the retriever returns insights about appreciation rates for quality homes, the relationship between living area and value, and investment considerations—all directly relevant to the advisory context.

15.3 Agent Workflow Effectiveness

The three-node agent workflow achieves its design goals:

- **Node 1 (Retrieval):** Successfully retrieves relevant market context from the vector store, providing the LLM with grounded evidence.
- **Node 2 (Analysis):** The ±20% valuation threshold provides a simple but effective heuristic for flagging over/undervalued properties relative to comparables.
- **Node 3 (Report Generation):** When the LLM is available, it produces well-structured, grounded advisory reports. When unavailable, the fallback generates consistent, rule-based reports that still provide actionable guidance.

15.4 Advisory Report Quality

The generated advisory reports follow a consistent structure:

1. **Property Valuation:** The ML-predicted price prominently displayed.
2. **Market Insights:** Three RAG-sourced insights relevant to the property.
3. **Comparable Analysis:** Summary of similar properties with valuation assessment.
4. **Recommendation:** One of Buy / Wait / Caution with supporting reasoning.
5. **Risks:** Two specific risk factors based on the valuation assessment.
6. **Disclaimer:** Standard advisory disclaimer.

Table 3 shows illustrative advisory outputs for different property profiles.

Table 3: Illustrative advisory outputs for different property profiles.

Profile	Predicted Price	Valuation	Recommendation
Starter (Qual=4, 1100 sqft)	∼$100K	Fair/Undervalued	Buy
Family (Qual=6, 1850 sqft)	∼$175K	Fair	Wait
Premium (Qual=8, 2850 sqft)	∼$310K	Fair/Overvalued	Caution

16 Limitations and Future Work

16.1 Current Limitations

- **Static knowledge base:** The RAG knowledge base consists of a small number of curated sentences. Real-time market data, news feeds, or regulatory updates are not incorporated.
- **LLM dependency:** The system relies on an external LLM API (OpenRouter). While the fallback mitigates this, the LLM-generated reports are richer and more nuanced.
- **Single dataset:** The model is trained on Ames, Iowa housing data and may not generalise to other markets without retraining.
- **Limited feature exposure:** Only 8 of the 79 features are exposed in the UI; the remaining features are zero-filled, which may affect prediction accuracy for properties that differ significantly in unexposed features.
- **No uncertainty quantification:** The ML model provides point estimates without confidence intervals.

16.2 Future Work

- **Richer RAG corpus:** Incorporate real-time property listings, market reports, and regulatory documents into the vector store.
- **Multi-agent validation:** Add a reviewer agent node that cross-checks the generated advisory against the evidence before returning the final report.
- **Advanced ML models:** Evaluate Gradient Boosting (XGBoost, LightGBM) and neural network approaches for potentially better predictive performance.
- **Explainability:** Integrate SHAP values or coefficient-based breakdowns to show which features most influenced each prediction.
- **Uncertainty bands:** Compute prediction intervals using validation residuals or conformal prediction methods.
- **Broader market coverage:** Extend to multiple housing markets with transfer learning or market-specific fine-tuning.

17 Additional Design Choices

17.1 Agent Design Choices

- **Linear DAG over dynamic routing:** A fixed three-node sequence provides predictable latency and deterministic behaviour, which is preferred for a user-facing application.
- **Prompt engineering:** The LLM prompt uses strict formatting rules and explicit instructions to avoid hallucination, ensuring the advisory stays grounded in the provided evidence.
- **Response caching:** LRU caching of LLM responses avoids redundant API calls for identical property configurations.
- **Text cleaning:** A dedicated text cleaner normalises LLM output (fixing spacing, formatting headings, cleaning bullet points) for consistent UI rendering.

17.2 Deployment Optimizations

- **Single artifact:** The entire preprocessing + model pipeline is packaged into one joblib file.
- **Version pinning:** Critical library versions are pinned in `requirements.txt` (scikit-learn 1.6.1, joblib 1.5.3, numpy 2.0.2, pandas 2.3.3).
- **Progress feedback:** A Streamlit progress bar provides real-time feedback during the multi-step analysis (15% → 35% → 55% → 80% → 100%).

18 Technology Stack

Table 4 summarises the complete technology stack.

Table 4: Technology stack and components.

Component	**Technology**
Language	Python 3.10+
ML Framework	Scikit-learn 1.6.1
Data Processing	Pandas 2.3.3, NumPy 2.0.2
Model Serialization	Joblib 1.5.3
Vector Store	ChromaDB
Embeddings	Sentence Transformers (all-MiniLM-L6-v2)
LLM Gateway	OpenRouter API
Agent Framework	LangGraph
RAG Framework	LangChain (Community + HuggingFace)
Web UI	Streamlit
Version Control	Git / GitHub

19 Team Contribution

The project was a collaborative effort. Each team member had clearly defined responsibilities, which are summarised below.

Kavya Katal

- Led **model training** and experimentation in the Jupyter notebooks.
- Designed and implemented the **preprocessing pipelines** using `ColumnTransformer` and `Pipeline`.
- Trained and evaluated the **Linear Regression** and **Random Forest** models.
- Contributed to the **technical report** and documentation of the modelling decisions.

Aabir Sarkar

- Handled **data cleaning and preprocessing**, including handling missing values, column type checks, and dataset sanity validation.
- Implemented the **Streamlit application** logic, including loading saved artifacts, building input DataFrames, and running predictions.
- Designed and implemented the **LangGraph agent workflow**, **RAG pipeline**, and **LLM integration**.
- Wrote and refined the **LaTeX report**, integrating results from the notebook and codebase.

Kushagra Maheswari

- Responsible for **dataset sourcing**, ensuring the correct Ames Housing dataset was obtained and integrated into the project.
- Created the **demo video** showcasing the model and the Streamlit app in action.
- Helped verify that the user interface behaved correctly for different input configurations.

Pratyush Chouksey

- Contributed to **dataset sourcing** and validation of the raw data.
- Assisted in writing parts of the **project report**, especially sections related to data description and EDA.
- Helped cross-check the consistency between the notebook results and the deployed model.

20 Conclusion

This project demonstrates how Machine Learning, Retrieval-Augmented Generation, and agentic workflows can be integrated into a cohesive real estate advisory system. The ML pipeline provides accurate price predictions ($R^2 = 0.93$), the RAG system grounds the advisory in retrievable domain knowledge, and the LangGraph agent orchestrates the multi-step reasoning required to produce structured, actionable investment reports.

The system's three-node agent architecture—Retrieve, Analyse, Generate—reflects a principled separation of concerns, while the rule-based fallback ensures reliability regardless of external API availability. Deployed as an interactive Streamlit application, the system makes the combined intelligence of statistical modelling and AI-augmented reasoning accessible to non-technical users.

Future directions include expanding the RAG knowledge base with real-time market data, adding multi-agent validation for report quality, integrating advanced ML models, and providing explainability through feature attribution methods.